

Corrigé — Examen d'entraînement LEPL1110 — Éléments Finis

Équipe pédagogique

Mai 2026

Exercice 1 — Équation de Poisson et éléments finis P_2

1.1 — Formulation faible

Dérivation. On multiplie $-u'' = f$ par $v \in V = H_0^1(0, 1)$ et on intègre :

$$-\int_0^1 u'' v \, dx = \int_0^1 f v \, dx.$$

Par intégration par parties :

$$-\int_0^1 u'' v \, dx = [-u' v]_0^1 + \int_0^1 u' v' \, dx.$$

Les termes de bord valent $-u'(1)v(1) + u'(0)v(0)$. Puisque $v \in H_0^1(0, 1)$, on a $v(0) = v(1) = 0$, donc les termes de bord **s'annulent**. On obtient :

$$a(u, v) = \ell(v) \quad \forall v \in V,$$

avec $a(u, v) = \int_0^1 u' v' \, dx$ et $\ell(v) = \int_0^1 f v \, dx$.

Conditions essentielles vs naturelles.

- Les **conditions de Dirichlet** ($u(0) = u(1) = 0$) sont dites **essentielles** : elles doivent être encodées dans l'espace $V = H_0^1(0, 1)$. Elles n'apparaissent pas explicitement dans la formulation faible.
- Les **conditions de Neumann** (p.ex. $u'(1) = g$) sont dites **naturelles** : elles apparaissent spontanément comme terme de bord dans $\ell(v)$ après intégration par parties, sans modifier l'espace V .

Erreur classique : Croire que les termes de bord s'annulent parce que $u' = 0$ en bord. En réalité, c'est $v = 0$ (car $v \in H_0^1$) qui les annule. La dérivée u' peut très bien être non nulle aux extrémités.

1.2 — Éléments P_2 de Lagrange

(a) **Fonctions de forme.** Les nœuds de l'élément de référence sont $\xi_1 = -1$, $\xi_2 = 0$, $\xi_3 = 1$:

$$N_1(\xi) = \frac{\xi(\xi - 1)}{2}, \quad N_2(\xi) = 1 - \xi^2, \quad N_3(\xi) = \frac{\xi(\xi + 1)}{2}.$$

Vérification ($N_i(\xi_j) = \delta_{ij}$) :

$$N_1(-1) = 1, N_1(0) = 0, N_1(1) = 0; \quad N_2(-1) = 0, N_2(0) = 1, N_2(1) = 0; \quad N_3(-1) = 0, N_3(0) = 0, N_3(1) = 1$$

(b) Dérivées et partition de l'unité.

$$N_1'(\xi) = \xi - \frac{1}{2}, \quad N_2'(\xi) = -2\xi, \quad N_3'(\xi) = \xi + \frac{1}{2}.$$

Partition de l'unité :

$$N_1 + N_2 + N_3 = \frac{\xi^2 - \xi}{2} + (1 - \xi^2) + \frac{\xi^2 + \xi}{2} = \xi^2 - \xi^2 + 1 = 1. \quad \checkmark$$

Erreur classique : Confondre N_1 et N_3 . Se rappeler que N_1 est la fonction égale à 1 au nœud *gauche* $\xi_1 = -1$: elle vaut $\frac{\xi(\xi-1)}{2}$ (s'annule en 0 et 1, vaut 1 en -1).

1.3 — Matrice de rigidité élémentaire P_2

(a) **Expression de $K_{ij}^{(e)}$.** Le changement de variable $x(\xi) = x_e + \frac{h}{2}(1 + \xi)$ donne $dx/d\xi = h/2$ et $d\xi/dx = 2/h$. La dérivée physique vaut :

$$\frac{dN_i}{dx} = \frac{2}{h} N_i'(\xi).$$

Donc :

$$K_{ij}^{(e)} = \int_{x_e}^{x_{e+1}} \frac{dN_i}{dx} \frac{dN_j}{dx} dx = \int_{-1}^1 \frac{2}{h} N_i'(\xi) \cdot \frac{2}{h} N_j'(\xi) \cdot \frac{h}{2} d\xi = \frac{2}{h} \int_{-1}^1 N_i'(\xi) N_j'(\xi) d\xi.$$

(b) **Calcul de $K_{11}^{(e)}$ et $K_{12}^{(e)}$.**

$$K_{11}^{(e)} = \frac{2}{h} \int_{-1}^1 \left(\xi - \frac{1}{2}\right)^2 d\xi = \frac{2}{h} \left[\frac{\xi^3}{3} - \frac{\xi^2}{2} + \frac{\xi}{4} \right]_{-1}^1 = \frac{2}{h} \left(\frac{2}{3} + \frac{1}{2} \right) = \frac{2}{h} \cdot \frac{7}{6} = \frac{7}{3h}.$$

$$K_{12}^{(e)} = \frac{2}{h} \int_{-1}^1 \left(\xi - \frac{1}{2}\right) (-2\xi) d\xi = \frac{2}{h} \left[-\frac{2\xi^3}{3} + \frac{\xi^2}{2} \right]_{-1}^1 = \frac{2}{h} \left(-\frac{4}{3} \right) = -\frac{8}{3h}.$$

(c) **Matrice complète.** Par symétrie $N_1 \leftrightarrow N_3$ (sous $\xi \mapsto -\xi$) : $K_{33}^{(e)} = K_{11}^{(e)}$, $K_{23}^{(e)} = K_{12}^{(e)}$.
Les entrées restantes :

$$K_{13}^{(e)} = \frac{2}{h} \int_{-1}^1 \left(\xi^2 - \frac{1}{4}\right) d\xi = \frac{2}{h} \cdot \frac{1}{6} = \frac{1}{3h}, \quad K_{22}^{(e)} = \frac{2}{h} \int_{-1}^1 4\xi^2 d\xi = \frac{2}{h} \cdot \frac{8}{3} = \frac{16}{3h}.$$

$$K^{(e)} = \frac{1}{3h} \begin{pmatrix} 7 & -8 & 1 \\ -8 & 16 & -8 \\ 1 & -8 & 7 \end{pmatrix}.$$

Vérification $K^{(e)} \mathbf{1} = \mathbf{0}$: $7 - 8 + 1 = 0$, $-8 + 16 - 8 = 0$, $1 - 8 + 7 = 0$. $\checkmark$

Interprétation : Un champ de température uniforme ($u = \text{cst}$) ne génère aucun flux. La rigidité annihile donc les vecteurs constants, ce qui traduit la conservation de l'énergie en l'absence de gradient.

1.4 — Application : $N = 1$ élément, $f = 2$

(a) **Fonctions physiques et vecteur charge.** Avec $\xi = 2x - 1$:

$$N_1(x) = (2x - 1)(x - 1), \quad N_2(x) = 4x(1 - x), \quad N_3(x) = x(2x - 1).$$

Calcul des composantes $F_i = 2 \int_0^1 N_i(x) dx$:

$$F_0 = 2 \int_0^1 (2x^2 - 3x + 1) dx = 2\left(\frac{2}{3} - \frac{3}{2} + 1\right) = 2 \cdot \frac{1}{6} = \frac{1}{3},$$

$$F_1 = 8 \int_0^1 x(1 - x) dx = 8 \cdot \frac{1}{6} = \frac{4}{3},$$

$$F_2 = 2 \int_0^1 (2x^2 - x) dx = 2\left(\frac{2}{3} - \frac{1}{2}\right) = 2 \cdot \frac{1}{6} = \frac{1}{3}.$$

$$\mathbf{F} = \left(\frac{1}{3}, \frac{4}{3}, \frac{1}{3}\right)^\top.$$

(b) **Système réduit.** Avec $h = 1$, $K = K^{(e)} = \frac{1}{3} \begin{pmatrix} 7 & -8 & 1 \\ -8 & 16 & -8 \\ 1 & -8 & 7 \end{pmatrix}$. Après élimination de

$u_0 = u_2 = 0$:

$$\frac{16}{3} u_1 = \frac{4}{3} \quad \implies \quad u_1 = \frac{1}{4}.$$

(c) **Comparaison.** $u(\frac{1}{2}) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$.

Résultat : $u_1 = u(\frac{1}{2})$ — la solution éléments finis est **exacte**.

Explication : La solution $u(x) = x(1 - x)$ est un polynôme de degré 2. Elle appartient à l'espace $V_h = P_2([0, 1])$. Par le lemme de Céa, $u_h = u$ exactement.

Erreur classique : Croire que l'exactitude vient du fait qu'il n'y a qu'un seul élément. La vraie raison est que $u \in V_h$, ce qui force l'erreur à zéro indépendamment du maillage.

1.5 — Lemme de Céa et convergence

(a) **Lemme de Céa.** Soit $a(\cdot, \cdot)$ bilinéaire, **continue** ($|a(u, v)| \leq M \|u\|_V \|v\|_V$) et **coercive** ($a(v, v) \geq \alpha \|v\|_V^2$). Soit $u_h \in V_h \subset V$ la solution de Galerkin. Alors :

$$\|u - u_h\|_V \leq \frac{M}{\alpha} \inf_{v_h \in V_h} \|u - v_h\|_V.$$

La solution discrète est **quasi-optimale** : elle réalise la meilleure approximation à la constante M/α près.

Pour l'équation de Poisson, $M = \alpha = 1$, donc $\|u - u_h\|_{H^1} \leq \inf_{v_h} \|u - v_h\|_{H^1}$.

(b) **Vitesses de convergence.** En combinant Céa et l'estimée d'interpolation :

$$\|u - u_h\|_{H^1} \leq C h^k |u|_{H^{k+1}}.$$

Élément	k	Convergence en H^1
P_1	1	$\mathcal{O}(h)$
P_2	2	$\mathcal{O}(h^2)$

Gain : Pour la même tolérance, P_2 permet d'utiliser un maillage environ $\sqrt{N_{P_1}/N_{P_2}}$ fois plus grossier. En pratique, pour une erreur cible ϵ : P_1 requiert $N \sim \epsilon^{-1}$ éléments, P_2 requiert seulement $N \sim \epsilon^{-1/2}$.

(c) Nombre minimal d'éléments. Condition : $h^2 \leq 10^{-4}$, soit $h \leq 10^{-2}$, donc :

$$N = \frac{1}{h} \geq 100.$$

Il faut au minimum $N = 100$ éléments P_2 .

Remarque : Avec P_1 et $|u|_{H^2} = 1$, il faudrait $h \leq 10^{-4}$, soit $N \geq 10\,000$ éléments — cent fois plus. Le passage de P_1 à P_2 est très avantageux dès que la solution est régulière ($u \in H^3$).

Erreur classique : Confondre convergence en H^1 (ordre k) et en L^2 (ordre $k + 1$, via l'astuce d'Aubin–Nitsche). Pour P_2 : erreur L^2 en $\mathcal{O}(h^3)$, erreur H^1 en $\mathcal{O}(h^2)$.

Exercice 2 — Corrigé : le jeu des erreurs

① — Mauvais nombre de nœuds dans linspace

```
1 x = np.linspace(0.0, 1.0, N + 1)      # ERREUR : N+1 noeuds (P1)
2 x = np.linspace(0.0, 1.0, 2*N + 1)    # CORRECT : 2N+1 noeuds pour
    P2
```

Pour N éléments P_2 , il faut $2N + 1$ nœuds (un nœud sommet par extrémité plus un nœud milieu par élément), et non $N + 1$.

② — Troisième DOF de l'élément incorrect

```
1 dof = [2*e, 2*e+1, 2*e+1]             # ERREUR : dernier noeud = noeud milieu
    (doublet)
2 dof = [2*e, 2*e+1, 2*e+2]             # CORRECT : noeuds gauche, milieu,
    droit
```

Le troisième degré de liberté de l'élément e est le nœud sommet droit $2e + 2$, pas le nœud milieu $2e + 1$ (qui est le deuxième DOF).

③ — Jacobien incorrect

```
1 Jac = h                               # ERREUR : longueur de l'element physique
2 Jac = h / 2.0                         # CORRECT : jacobien de l'application reference
    [-1, 1] -> [x_e, x_e+h]
```

Le changement de variable $x(\xi) = x_e + \frac{h}{2}(1 + \xi)$ a pour jacobien $dx/d\xi = h/2$, et non h .

④ — Dérivée de N_2 incorrecte

```
1 dN_dxi = [xi - 0.5, 2.0 * xi, xi + 0.5] # ERREUR : N2'(xi) = +2
    xi
2 dN_dxi = [xi - 0.5, -2.0 * xi, xi + 0.5] # CORRECT : N2'(xi) =
    -2 xi
```

La dérivée de $N_2(\xi) = 1 - \xi^2$ est $N_2'(\xi) = -2\xi$, pas $+2\xi$.

⑤ — Mauvais sens du jacobien inverse

```
1 dxi_dx = h / 2.0                     # ERREUR : c'est dx/dxi (le jacobien), pas dxi/
    dx
2 dxi_dx = 2.0 / h                     # CORRECT : dxi/dx = 2/h (inverse du jacobien)
```

On a $d\xi/dx = 2/h$. La valeur $h/2$ est le jacobien direct $dx/d\xi$, l'opposé de ce qu'on cherche.

⑥ — Indice J figé sur la ligne i

```
1 J = dof[i]                           # ERREUR : J ne change pas avec j -> seule la
    diagonale est remplie
2 J = dof[j]                           # CORRECT
```

La boucle interne porte sur j , donc l'indice de colonne doit être `dof[j]`.

⑦ — Retour de valeur incorrect

```
1 return x, K      # ERREUR : retourne aussi le tableau x (non demande)
2 return K         # CORRECT : seule la matrice K est attendue
```

La spécification demande uniquement la matrice K . Retourner le tuple (x, K) causera des erreurs dans le code appelant.

Code corrigé complet

```
1 import numpy as np
2
3 def assemble_stiffness_p2(N):
4     """Matrice de rigidite P2 sur (0,1), N elements uniformes, sans
5         CLs."""
6     n_nodes = 2 * N + 1
7     h = 1.0 / N
8
9     x = np.linspace(0.0, 1.0, 2 * N + 1)      # corrige (1)
10
11     K = np.zeros((n_nodes, n_nodes))
12
13     xi_q = [-np.sqrt(3.0 / 5.0), 0.0, np.sqrt(3.0 / 5.0)]
14     w_q = [5.0 / 9.0, 8.0 / 9.0, 5.0 / 9.0]
15
16     for e in range(N):
17         dof = [2 * e, 2 * e + 1, 2 * e + 2]    # corrige (2)
18         Jac = h / 2.0                          # corrige (3)
19
20         for q in range(3):
21             xi = xi_q[q]
22             w = w_q[q]
23
24             dN_dxi = [xi - 0.5,
25                     -2.0 * xi,
26                     xi + 0.5]                  # corrige (4)
27
28             dxi_dx = 2.0 / h                   # corrige (5)
29
30             dN_dx = [dN_dxi[k] * dxi_dx for k in range(3)]
31
32             for i in range(3):
33                 I = dof[i]
34                 for j in range(3):
35                     J = dof[j]
36                     K[I, J] += dN_dx[i] * dN_dx[j] * w * Jac
37                     # corrige (6)
38
39     return K                                  # corrige (7)
```